

JAVA 17 OCP · YDS STUDY SERIES

Unit 01 · Technical Memory Notes · Building Blocks

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya eksiksiz [çift dilli ana notu](#) tamamlayan teknik hafıza katmanıdır. Amaç, ayrıntıları tek tek ezberlemek yerine Java programının kaynak kod → derleme → class initialization → object initialization → reachability akışını zihinde sabitlemektir. Dil çalışması için [vocabulary](#) kaynağına bakılabilir.

Kapsam denetimi

Ana çift dilli kaynak environment, class yapısı, `main()`, packages/imports, primitive types, `var`, scope ve garbage collection konularını tam kapsar. Bu hafıza notu özellikle şu yüksek riskli bağları birlikte tekrar ettirir:

- local variable default değeri ile field default değerinin ayrılması,
- declaration scope ile object lifetime'ın aynı şey olmadığına görülmesi,
- static/instance initialization sırasının tek akışta izlenmesi,
- `var`'ın dynamic typing olmadığı ve inferred type'ın değişmediğinin hatırlanması.

1. Büyük resim: dört durak

```
.java source
  ↓ javac
.class bytecode
  ↓ java / JVM class loading
static initialization (bir class için bir kez)
  ↓ new
instance initialization (her object için yeniden)
```

Hafıza cümlesi: Kaynak derlenir, class bir kez hazırlanır, object her `new` işleminde yeniden kurulur.

- `javac` compilation yapar; syntax ve type hataları burada bulunur.
- `java` JVM'i başlatıp belirtilen class'ın uygun `main()` method'unu arar.
- JVM aynı bytecode'u desteklenen platformlarda çalıştırdığı için Java portable'dır.
- JDK; compiler, runtime araçları ve geliştirme araçlarını içerir.

2. Class dosyasının kapı sırası

```
package zoo.animals;           // varsa ilk Java declaration

import java.util.List;         // package'tan sonra

public class Lion {             // public top-level type dosya adıyla eşleşir
    private int age;           // member'lar class body içindedir
}
```

Bir source file yalnız bir `public` top-level type taşıyabilir. Dosyada birden fazla package-private top-level type olabilir. Import wildcard yalnız aynı package içindeki type'ları kapsar; subpackage'leri kapsamaz.

```
import java.util.*;           // java.util.ArrayList var
// java.util.concurrent.* otomatik gelmez
```

Conflict sırası: Explicit import, wildcard'dan daha belirgindir; fakat aynı simple name için iki explicit import compilation error üretir. `java.lang` otomatik import edilir, current package type'ları da import gerektirmez.

3. `main()` giriş noktası

Aşağıdaki imzalar alternatif yazımlardır; aynı class içinde birlikte bildirilirlerse aynı signature nedeniyle derlenmezler.

```
public static void main(String[] args) {}
public static void main(String... args) {} // eşdeğer parameter biçimi
```

- `public` ve `static` gereklidir; sıraları değişebilir.
- Return type tam olarak `void` olmalıdır.
- Parameter type `String[]` veya `String...` olabilir; adı serbesttir.
- `String args[]` geçerli olsa da sınavda daha az yaygın görünür.
- Overloaded başka `main()` method'ları entry point değildir.

4. Variable karar tablosu

Tür	Nerede bildirilir?	Default değer	Kullanımdan önce initialization
Local variable	method/block içinde	Yok	Zorunlu
Parameter	method/constructor header	Caller verir	Otomatik
Instance field	class body, <code>static</code> değil	Var	Zorunlu değil
Static field	class body, <code>static</code>	Var	Zorunlu değil

Field default'ları: numeric types `0`, `char` `\u0000`, `boolean` `false`, reference types `null`.

```
class Defaults {
    int field;                // 0

    void test() {
        int local;
        System.out.println(field);
        // System.out.println(local); // Does not compile
    }
}
```

Hafıza cümlesi: Field bekleyebilir; local variable bekleyemez.

5. Primitive, reference ve literal kontrolü

Java'nın sekiz primitive type'ı: `boolean`, `byte`, `short`, `int`, `long`, `char`, `float`, `double`. Bunların dışındaki variable type'lar reference type'dır.

- Integer literal varsayılan olarak `int`, decimal literal `double`dir.

- `long` için `L`, `float` için `F` suffix'i kullanılır.
- `_` digit'ler arasında olabilir; literal'in başında, sonunda veya decimal point'in hemen yanında olamaz.
- `char` unsigned 16-bit integral type'dır; single quote kullanır.
- Reference variable `null` olabilir; primitive olamaz.

6. `var`: type gizlenmez, yalnız yazılmaz

```
var count = 3;           // int
var name = "lion";       // String
final var limit = 10;    // geçerli
```

Normal local variable declaration'da `var` kullanıldığında aynı declaration içinde initializer gerekir. Enhanced `for` variable'ı ve lambda parameter'ı gibi özel bağlamlarda ayrıca kurallar vardır; `var` field, return type veya sıradan method parameter type'ı olamaz. Aşağıdakiler derlenmez:

```
// var missing;
// var empty = null;
// var values = {1, 2, 3};
// public var method() { return 1; }
```

`var x = 3;` sonrasında `x` her zaman `int`'tir. Başka type assignment'ı dynamic olarak type değiştirmez.

Hafıza cümlesi: `var` type'ı silmez; compiler'a bir kez buldurur.

7. Initialization sırası

Bir class ilk kez aktif kullanıldığında static fields ve static initializer blocks source order ile bir kez çalışır. Her `new` işleminde:

- 1 object fields default değerlerini alır,
- 2 instance field initializers ve instance initializer blocks source order ile çalışır,
- 3 constructor body çalışır.

Inheritance varsa static initialization superclass'tan subclass'a ilerler. Object alanları önce default değerlerini alır; ardından constructor zincirinde superclass'ın instance initialization'ı ve constructor gövdesi, subclass'ın instance initialization'ından önce tamamlanır. Ayrıntılı constructor zinciri için [Unit 06 teknik hafıza notuna](#) bak.

8. Scope, lifetime ve garbage collection

Bir reference variable scope dışına çıktığında ona erişilemez; fakat gösterdiği object başka bir reachable reference tarafından tutuluyorsa yaşamaya devam eder. Object ancak GC roots'tan hiçbir reference yolu kalmadığında eligible for garbage collection olur.

```
var a = new StringBuilder("cat");  
var b = a;  
a = null;           // object hâlâ b üzerinden reachable  
b = null;           // artık eligible; ne zaman silineceği garanti edilmez
```

`System.gc()` yalnız request'tir; collection zamanını garanti etmez.

OCP için 20 saniyelik analiz sırası

- 1 Kod parse ediliyor mu; declaration sırası ve isimler geçerli mi?
- 2 Variable local mı field mı; initialization zorunlu mu?
- 3 Inferred/declared type assignment ile uyumlu mu?
- 4 Static initialization mı, instance initialization mı çalışıyor?
- 5 GC sorusuysa object'e giden başka reference yolu var mı?

Aktif hatırlama · Özgün çalışma soruları

- 1 `var x = null;` neden derlenmez?
- 2 `int value;` declaration'ı field ve local olduğunda fark nedir?
- 3 Bir object'e iki reference bakarken yalnız biri `null` yapılırsa object GC'ye uygun olur mu?
- 4 `import java.util.*;` ifadesi `java.util.concurrent.TimeUnit` type'ını getirir mi?

Cevaplar

- 1 Compiler initializer'dan type çıkaramaz.
- 2 Field `0` alır; local variable kullanımdan önce explicit initialize edilmelidir.
- 3 Hayır; diğer reachable reference object'i canlı tutar.
- 4 Hayır; wildcard subpackage'leri kapsamaz.